# DAR ES SALAAM INSTITUTE OF TECHNOLOGY

# A MULTI-FACTOR AUTHENTICATED DIGITAL ATTENDANCE SYSTEM

## WITH PROXY-PREVENTION MECHANISM AND REAL-TIME LECTURER MONITORING

*A Project Report Submitted in Partial Fulfilment of the Requirements for the Module: Digital Electronics (ETU 07322)*

**Module Master: Dr. Georgia Rugumira**
**Class: BENG24ETE-3 | Group No: 06**


| S/N | FULL NAME | REGISTRATION NO. | MARKS |
|---|---|---|---|
| 1 | Lameck Halle Mbetwa | 240647465425 | |
| 2 | Kelvin Pantaleo Massawe | 240647438059 | |
| 3 | Isack Festo Lubigisa | 240647414043 | |
| 4 | Faida Sylivester Mosses | 240647474625 | |
| 5 | Brenda Edward Kimaro | 240647474914 | |
| 6 | Debora Nelson Moshi | 240647455798 | |
| 7 | John Julius John | 240647487791 | |

## ABSTRACT

This paper presents the design and implementation of a Multi-Factor Authenticated Digital Attendance System with a built-in proxy-prevention mechanism and real-time lecturer monitoring capability, developed using an Arduino Uno microcontroller platform. The system was developed as part of the Digital Electronics module (ETU 07322) at Dar es Salaam Institute of Technology (DIT). The proposed system addresses the persistent problem of proxy attendance in academic institutions by employing three-factor authentication: a six-digit registration identifier, a private four-digit Personal Identification Number (PIN) derived from the student's date of birth, and a session-specific One-Time Password (OTP) generated at the start of each class.

The system incorporates a physical attendance count limit set by the lecturer, which causes the system to lock immediately upon reaching the confirmed number of physically present students, thereby making proxy attendance self-defeating. A Finite State Machine (FSM) architecture governs all system transitions, representing the sequential circuit component of the design. Combinational circuit principles are demonstrated through the ATmega328P's internal ALU performing OTP comparison and keypad decoding logic. The system outputs a complete real-time attendance log to the lecturer's Serial Monitor, listing present and absent students with timestamps. Hardware components include a 4x4 matrix keypad, I2C 16x2 LCD display, active buzzer, LED indicator, and Arduino Uno. The system was tested with seven students and demonstrated effective authentication, proxy detection, and automatic session locking.


**Keywords:** *Digital Attendance, Multi-Factor Authentication, OTP, Proxy Prevention, Arduino, Finite State Machine, Combinational Circuit, Sequential Circuit, Embedded Systems, ATmega328P.*

# I. INTRODUCTION

Attendance management is a fundamental administrative function in academic institutions. Traditional paper-based attendance registers are prone to manipulation, particularly the practice known as proxy attendance, where one student signs on behalf of an absent colleague. This undermines academic integrity, distorts performance records, and reduces institutional accountability [1].

Existing electronic solutions, such as biometric fingerprint systems and RFID-based systems, address proxy attendance effectively but often require significant financial investment, internet connectivity, and complex infrastructure that may be unavailable in resource-constrained environments such as technical colleges and laboratories in developing countries [2].

This project presents a cost-effective, standalone, offline solution developed entirely using digital electronics principles applied through the Arduino Uno platform. The system demonstrates the practical application of combinational and sequential circuit theory by implementing three-factor authentication, a session-based OTP mechanism, a physical count limit with automatic locking, and a real-time attendance log delivered through the Arduino Serial Monitor to the lecturer's computer.

The system was designed, built, and tested by Group 06 of Class BENG24ETE-3 at Dar es Salaam Institute of Technology as a project submission for the Digital Electronics module (ETU 07322) under the supervision of Dr. Georgia Rugumira.

# II. PROBLEM STATEMENT

The core problems identified in conventional attendance systems used in academic institutions include the following. First, proxy attendance is easily executed in paper-based systems since any student can sign on behalf of an absent colleague without any technical barrier. Second, static password systems, if employed digitally, are vulnerable to credential sharing since passwords do not change between sessions. Third, manual reconciliation of attendance registers is time-consuming and error-prone. Fourth, existing biometric solutions are expensive and require internet connectivity or dedicated servers, making them unsuitable for small institutions with limited budgets.

These problems collectively result in unreliable attendance records, reduced accountability, and compromised academic integrity. The system presented in this paper specifically addresses all four problems within the constraints of a low-cost embedded hardware platform.

# III. OBJECTIVES

## A. General Objective

To design and implement a multi-factor authenticated digital attendance system using an Arduino-based embedded platform that prevents proxy attendance and provides real-time monitoring for lecturers.

### B. Specific Objectives

1. To implement three-factor authentication combining registration number, personal PIN, and session OTP.

2. To design a Finite State Machine (FSM) to govern all system state transitions.

3. To implement a physical count limit mechanism that locks the system automatically upon reaching the lecturer-specified attendance threshold.

4. To provide real-time attendance logging on the lecturer's Serial Monitor with student names, status, and time taken.

5. To demonstrate the presence of combinational and sequential circuits within the Arduino-based implementation.

6. To justify the use of a microcontroller platform over discrete logic gate implementations.

## IV. LITERATURE REVIEW

Authentication systems have evolved significantly over the decades. Early systems relied on static passwords, which are vulnerable to sharing and replay attacks. One-Time Password (OTP) systems were introduced to overcome these weaknesses by generating a unique credential valid for only one session [3].

Lamport [4] first proposed the concept of one-time passwords in 1981 as a method of authenticating users over insecure networks. Modern OTP systems are categorised into Time-based OTP (TOTP) and HMAC-based OTP (HOTP), both standardised under RFC 6238 and RFC 4226 respectively [5]. These systems rely on shared secrets and cryptographic hash functions and typically require server infrastructure and internet connectivity.

In the context of attendance management, biometric systems using fingerprint sensors such as the AS608 module have been shown to be highly effective at preventing proxy attendance [6]. However, their cost and requirement for prior enrollment make them difficult to deploy rapidly in resource-constrained environments.

RFID-based attendance systems have also been widely researched. These systems use unique card identifiers assigned to each student. While efficient, RFID cards can be passed between students, making them susceptible to proxy attendance unless combined with additional verification [7].

Microcontroller-based embedded systems offer a middle ground: they are low-cost, offline-capable, and sufficiently flexible to implement multi-factor authentication logic in software while the underlying hardware performs the actual combinational and sequential circuit operations internally [8]. The ATmega328P microcontroller used in the Arduino Uno integrates an ALU, registers, timers, SRAM, and EEPROM, all of which represent functional combinational and sequential circuit components operating at the semiconductor level [9].

The Finite State Machine (FSM) model, a fundamental concept in digital electronics, has been applied in numerous embedded system designs to manage complex input-output relationships in a structured and predictable manner [10]. The use of FSM in this project aligns the software design directly with sequential circuit theory as taught in digital electronics curricula.

## V. JUSTIFICATION FOR USING ARDUINO UNO OVER DISCRETE LOGIC GATES

A central question in this project is why the Arduino Uno microcontroller was chosen as the implementation platform rather than building the circuit using discrete components such as flip-flops, logic gates, multiplexers, and counters. This section provides a thorough justification from both practical and academic perspectives.

### A. Discrete Logic Gate Implementation — What It Would Require

To implement this attendance system using only discrete digital components, the following would be required at minimum:

A 4-bit or 6-bit binary comparator built from XOR and AND gates to compare entered digits against stored values. Multiple D flip-flops or JK flip-flops to store the generated OTP, the entered PIN, and the current system state. A counter circuit made from cascaded flip-flops to track the number of students who have signed. A clock oscillator circuit to drive all sequential elements. Multiplexers to route keypad row and column signals. A ROM or EPROM chip to store student registration data. A display driver circuit to interface with the LCD. Separate debounce circuits for every keypad key.

Conservatively, such a system would require over 200 individual integrated circuit packages on a large PCB, costing significantly more than the Arduino-based solution, and taking months to design, fabricate, and debug. Any modification to the authentication logic would require physical circuit redesign.

### B. Arduino Contains All These Components Internally

The Arduino Uno uses the ATmega328P microcontroller, which integrates all the above discrete components on a single silicon chip at the transistor level. Specifically:

The Arithmetic Logic Unit (ALU) inside the ATmega328P is built from AND, OR, XOR, and NOT gates and performs all comparison operations. This is the combinational circuit. The 32 general-purpose registers, 2KB of SRAM, and 1KB of EEPROM are built from flip-flop arrays and constitute the sequential memory elements. Three hardware timers built from counters and flip-flops provide the clock-driven sequential behaviour. The 16 MHz crystal oscillator provides the master clock signal that drives all sequential operations, exactly as a clock circuit would in a discrete design.

Therefore, using Arduino does not mean avoiding digital electronics. It means using a highly integrated realisation of the same combinational and sequential circuits, implemented at the semiconductor level inside the ATmega328P chip.

### C. Practical Advantages of the Arduino Approach

| Criterion | Discrete Logic Gates | Arduino Uno (ATmega328P) |
|---|---|---|
| **Cost** | High (200+ ICs) | Low (~$5 board) |
| **Physical Size** | Very large PCB | Small single board |
| **Development Time** | Months | Days |
| **Modifiability** | Requires PCB redesign | Change code only |
| **Reliability** | Many solder joints | Single chip, robust |
| **Power Consumption** | High | Low (~50mA) |
| **Student Data Storage** | Requires EPROM + decoder | Built-in SRAM/EEPROM |
| **FSM Implementation** | Complex flip-flop network | Simple enum in code |
| **Combinational Logic** | Physical gate ICs | Internal ALU gates |
| **Clock Circuit** | Separate oscillator IC | Built-in 16MHz crystal |

### D. Academic Justification

From a digital electronics theory perspective, the Arduino-based implementation satisfies all requirements of the module because the same fundamental circuits are present, just integrated rather than discrete. The ATmega328P is fabricated using CMOS technology, meaning every logic operation ultimately reduces to transistor-level NOT, NAND, and NOR gate combinations, which are the universal building blocks of all digital electronics. Choosing Arduino is therefore not a shortcut around digital electronics. It is an application of digital electronics at the system integration level, which is the direction modern engineering practice has taken since the development of VLSI technology in the 1980s.

## VI. COMBINATIONAL AND SEQUENTIAL CIRCUITS IN THE SYSTEM

This section identifies and explains where combinational and sequential circuit principles are present within the implemented system, both at the hardware level inside the ATmega328P and at the logical level in the software design.

### A. Combinational Circuits

A combinational circuit is one whose output depends only on the current inputs, with no memory of previous states. The following combinational circuit functions are present in this system:

1. OTP Comparison (Digital Comparator): When the student enters the session OTP, the microcontroller's ALU compares the entered string against the stored sessionOTP variable bit by bit using XOR operations. If all bits match, the XOR output is zero, and an AND gate confirms equality. This is the functional equivalent of a 6-digit digital comparator built from XOR and AND gates.

2. PIN Comparison: Similarly, the entered birthday PIN is compared against the stored PIN for the identified student using the same ALU comparator logic. Output depends only on the two current inputs being compared.

3. Keypad Decoding: The 4x4 keypad produces a unique combination of row and column signals for each key press. The decoding of these signals into a character value is a combinational function, equivalent to a priority encoder or decoder implemented in gate logic.

4. LED and Buzzer Activation: The LED turns on immediately when authentication succeeds and off when it fails. The buzzer sounds based on the current authentication result. Both outputs depend only on the present authentication outcome — no memory is involved. This is purely combinational behaviour.

### B. Sequential Circuits

A sequential circuit is one whose output depends on both the current input and the previous state, requiring memory elements such as flip-flops driven by a clock signal.

1. Finite State Machine (FSM): The system's state machine is the primary sequential circuit in this design. It maintains seven states: S_IDLE, S_ENTER_COUNT, S_READY, S_ENTER_REG, S_ENTER_PIN, S_ENTER_OTP, and S_DONE. The current state is stored in a register (implemented as a variable in SRAM, which is built from flip-flops). State transitions depend on both the current state and the current input, matching the exact definition of a Moore-type FSM, which is a sequential circuit.

2. Student Data Storage: The registration numbers, PINs, and names of all seven students are stored in the ATmega328P's SRAM. SRAM cells are built from cross-coupled NAND or NOR gate latches, which are the fundamental sequential storage elements. The data persists across clock cycles because of this flip-flop-based storage.

3. Attendance Counter: The presentCount variable increments by one each time a student successfully authenticates. A counter that depends on its previous value to produce its next output is the textbook definition of a sequential circuit, implemented in discrete designs using cascaded JK or D flip-flops clocked by a common signal.

4. Timer Operation: The 15-second per-student timeout is implemented using the millis() function, which reads from the ATmega328P's Timer0 hardware counter. Timer0 is a sequential circuit built from an 8-bit binary ripple counter made of flip-flops, driven by the 16 MHz clock signal.

5. Clock-Driven Execution: Every instruction executed by the ATmega328P is synchronised to the 16 MHz crystal oscillator. This clock drives all flip-flops, registers, and counters throughout the chip, making the entire microcontroller a large synchronous sequential system.

| System Function | Circuit Type | Reason |
|---|---|---|
| OTP Comparison | **Combinational** | Output depends only on present inputs |
| PIN Comparison | **Combinational** | Immediate comparison, no history needed |
| Keypad Decoding | **Combinational** | Fixed mapping of row/col to character |
| LED/Buzzer Output | **Combinational** | Directly reflects current auth result |
| Finite State Machine | **Sequential** | Stores and transitions between states |
| SRAM Data Storage | **Sequential** | Flip-flop cells retain data across cycles |
| Attendance Counter | **Sequential** | Output depends on previous count value |
| Timeout Timer | **Sequential** | Hardware counter driven by 16MHz clock |

# TRUTH TABLES — COMBINATIONAL & SEQUENTIAL CIRCUITS

*Multi-Factor Authenticated Digital Attendance System (4×3 Keypad Version)*

## PART A — COMBINATIONAL CIRCUIT TRUTH TABLES

Combinational circuits produce outputs that depend only on present inputs, with no memory of previous states. The following circuits are present inside the ATmega328P ALU and in the system logic.

### A1. 1-Bit Comparator Gate (XOR / XNOR) — Building Block

The fundamental comparison unit. For each digit position, one XOR gate compares bit A (entered) against bit B (stored). A MATCH on all digit positions produces EQ = 1.

| A (Bit i) | B (Bit i) | XOR (A⊕B) | XNOR (Match) |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

*Note: Boolean expression: EQ = NOT(A XOR B) = XNOR(A, B). Six such gates in parallel implement the 6-digit OTP comparator.*

### A2. 6-Digit OTP Comparator (Digital Comparator)

Six XOR gates compare the entered OTP against the stored sessionOTP digit-by-digit. A NOR gate combines all XOR outputs — EQ = 1 only when ALL six XOR outputs are 0 (all digits match). This is functionally equivalent to a 6-digit digital comparator.

| Entered OTP (E) | Stored OTP (S) | Comparison | XOR5 | XOR4 | XOR3 | XOR2 | XOR1 | XOR0 | EQ (NOR of XORs) | Output / Action |
|---|---|---|---|---|---|---|---|---|---|---|
| 000000 | 000000 | Match | 0 | 0 | 0 | 0 | 0 | 0 | 1 | Marked PRESENT |
| 748291 | 748291 | Match | 0 | 0 | 0 | 0 | 0 | 0 | 1 | Marked PRESENT |
| 123456 | 654321 | Mismatch | 1 | 0 | 0 | 0 | 0 | 0 | 0 | Wrong OTP |
| 748291 | 748290 | Mismatch | 0 | 0 | 0 | 0 | 0 | 1 | 0 | Wrong OTP |
| 000000 | 000001 | Mismatch | 0 | 0 | 0 | 0 | 0 | 1 | 0 | Wrong OTP |
| 999999 | 999999 | Match | 0 | 0 | 0 | 0 | 0 | 0 | 1 | Marked PRESENT |

*Note: EQ = NOR(XOR5, XOR4, XOR3, XOR2, XOR1, XOR0). EQ = 1 → student marked PRESENT. EQ = 0 → Wrong OTP displayed.*

### A3. 4-Digit Birthday PIN Comparator

Same architecture as the OTP comparator, but 4 digits wide. The entered birthday PIN (DDMM format) is compared against the student's stored PIN. EQ = 1 only when all 4 digit pairs match.

| Entered PIN (E) | Stored PIN (S) | XOR3 | XOR2 | XOR1 | XOR0 | EQ (NOR) | Action |
|---|---|---|---|---|---|---|---|
| 1503 | 1503 | 0 | 0 | 0 | 0 | 1 | PIN Accepted → OTP stage |
| 2712 | 2712 | 0 | 0 | 0 | 0 | 1 | PIN Accepted → OTP stage |
| 1234 | 1503 | 1 | 0 | 0 | 1 | 0 | Wrong PIN → Access Denied |
| 0000 | 1503 | 1 | 1 | 0 | 1 | 0 | Wrong PIN → Access Denied |
| 9999 | 0412 | 1 | 0 | 1 | 1 | 0 | Wrong PIN → Access Denied |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0412 | 0412 | 0 | 0 | 0 | 0 | 1 | PIN Accepted → OTP stage |

*Note: EQ = NOR(XOR3, XOR2, XOR1, XOR0). EQ = 1 → proceed to OTP stage. EQ = 0 → return to READY state.*

### A4. 4×3 Matrix Keypad Decoder

The keypad library scans rows (R0–R3, pins 9-8-7-6) and columns (C0–C2, pins 5-4-3). For each unique active row–column pair, a fixed character is output. This is a pure combinational function equivalent to a priority encoder. No memory is involved.

| R0 | R1 | R2 | R3 | C0 | C1 | C2 | Key Output |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 1 | 0 | 0 | '1' |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 | '2' |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | '3' |
| 0 | 1 | 0 | 0 | 1 | 0 | 0 | '4' |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | '5' |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 | '6' |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 | '7' |
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | '8' |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | '9' |
| 0 | 0 | 0 | 1 | 1 | 0 | 0 | '*' |
| 0 | 0 | 0 | 1 | 0 | 1 | 0 | '0' |
| 0 | 0 | 0 | 1 | 0 | 0 | 1 | '#' |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | No key (idle) |

*Note: Rows: R0=pin 9, R1=pin 8, R2=pin 7, R3=pin 6. Columns: C0=pin 5, C1=pin 4, C2=pin 3. Only one row–column pair is active at a time. Multiple simultaneous presses are undefined.*

### A5. LED and Buzzer Output Logic

The LED (pin 13) and buzzer (pin 10) are driven directly from the authentication result. Their output depends only on the current authentication outcome — no memory is involved, making this purely combinational.

| Auth Result (Input) | LED Pin 13 | LED State | Buzzer (Pin 10) | LCD Output |
|---|---|---|---|---|
| OTP Match | 1 | ON (2 s) | Double beep (1.2 kHz + 1.6 kHz) | PRESENT |
| OTP Mismatch | 0 | OFF | Long beep (300 Hz, 600 ms) | Wrong OTP |
| PIN Mismatch | 0 | OFF | Long beep (300 Hz, 600 ms) | Access Denied |
| Reg Not Found | 0 | OFF | Long beep (300 Hz, 600 ms) | Not Found |
| Session Start | X | OFF | Triple ascending beep | OTP announced |
| Session Lock | X | OFF | Triple 1.8 kHz beep | Session closed |

*Note: X = don't care. LED uses digitalWrite(); Buzzer uses tone(). Both outputs are determined entirely by the present authentication result.*

## PART B — SEQUENTIAL CIRCUIT TRUTH TABLES

Sequential circuits produce outputs that depend on both current inputs AND previous state (memory). They require clock-driven flip-flops to store state. The following sequential elements are present in the ATmega328P and in the system logic.

## B1. Finite State Machine (FSM) — State Transition Table

The FSM is a Moore-type sequential circuit with 7 states stored in SRAM (flip-flop array). The next state depends on both the current state and the current input. This is the primary sequential circuit of the system.

| Current State (Q) | Input / Condition | Next State (Q+) | Output / LCD Action |
|---|---|---|---|
| S_IDLE | # pressed | S_ENTER_COUNT | Blank / How many here? |
| S_IDLE | Any other key | S_IDLE | No change |
| S_ENTER_COUNT | Digit (0–9) | S_ENTER_COUNT | Append to buffer |
| S_ENTER_COUNT | # (buffer non-empty) | S_READY | Generate OTP, show progress |
| S_ENTER_COUNT | * (backspace) | S_ENTER_COUNT | Delete last digit |
| S_READY | # pressed | S_ENTER_REG | Start 30 s timer |
| S_READY | ** (double-star) | S_DONE | End session → report |
| S_ENTER_REG | Digit (6 entered) + # | S_ENTER_PIN | Reg found & not signed |
| S_ENTER_REG | Digit (6 entered) + # | S_READY | Reg not found / duplicate |
| S_ENTER_REG | * (empty buffer) | S_READY | Cancel → back to ready |
| S_ENTER_REG | Timeout (30 s) | S_READY | Time Expired! |
| S_ENTER_PIN | 4 digits + # (correct) | S_ENTER_OTP | Enter OTP stage |
| S_ENTER_PIN | 4 digits + # (wrong) | S_READY | Wrong PIN → Access Denied |
| S_ENTER_PIN | * (empty buffer) | S_ENTER_REG | Back to reg entry |
| S_ENTER_PIN | Timeout (30 s) | S_READY | Time Expired! |
| S_ENTER_OTP | 6 digits + # (correct) | S_READY | PRESENT, count++ |
| S_ENTER_OTP | 6 digits + # (correct, count=physical) | S_DONE | LIMIT REACHED → lock |
| S_ENTER_OTP | 6 digits + # (wrong) | S_READY | Wrong OTP → See Lecturer |
| S_ENTER_OTP | * (empty buffer) | S_ENTER_PIN | Back to PIN entry |
| S_ENTER_OTP | Timeout (30 s) | S_READY | Time Expired! |
| S_DONE | Report printed (4 s) | S_IDLE | Press # to start new session |

*Note: State variable stored in SRAM — a flip-flop-based memory element clocked at 16 MHz. Moore machine: outputs depend on current state only. X = don't care. ** = double-star (two * presses within 600 ms).*

## B2. D Flip-Flop — Student signed_in[ ] Storage Cell

Each student has one bit in the signed_in[] boolean array stored in SRAM. SRAM cells are built from cross-coupled NAND gate latches (D flip-flops). When a student is marked PRESENT, D=1 is written; the bit persists across clock cycles until session reset.

| D (Data In) | CLK | EN (Enable) | Q (Next) | Q̄ (Next) | System Meaning |
|---|---|---|---|---|---|
| 0 | ↑ | 0 | 0 | 1 | Hold 0 — student absent |
| 0 | ↑ | 1 | 1 | 0 | Store 1 — student marked present |
| 1 | ↑ | 0 | 0 | 1 | Store 0 (reset between sessions) |
| 1 | ↑ | 1 | 1 | 0 | Hold 1 — already present |
| X | 0 | X | Q | Q̄ | No clock edge — state held |

*Note: CLK ↑ = rising edge of 16 MHz clock. EN = write enable (triggered on successful OTP match). 7 such flip-flops exist, one per student. At session start, all Q outputs are reset to 0.*

## B3. Attendance Counter (presentCount) — Synchronous Binary Counter

The presentCount variable acts as a synchronous binary counter. Its next value depends on its current value (previous state), making it a sequential circuit. In discrete logic this would be implemented using cascaded JK or D flip-flops. The table below shows a 3-bit example (for a class of up to 7 students).

| Q2 Q1 Q0 (Count) | RESET | Q2+ Q1+ Q0+ (Next Count) | System Action |
|---|---|---|---|
| 000 (0) | 0 | 000 (0) | Continue — no lock |
| 001 (1) | 0 | 001 (1) | Continue — no lock |
| 010 (2) | 0 | 010 (2) | Continue — no lock |
| 011 (3) | 0 | 011 (3) | Continue — no lock |
| 100 (4) | 0 | 100 (4) | Continue — no lock |
| 101 (5) | 0 | 101 (5) | Continue — no lock |
| 110 (6) | 0 | 110 (6) | Continue — no lock |
| 111 (7) | 0 | 111 (7) | Count = Physical Count → LOCK |
| Any | 1 | 000 (0) | Session reset — counter cleared |

*Note: Q2 Q1 Q0 are the three bits of presentCount. INCREMENT is triggered on each successful OTP match. RESET occurs when a new session starts (A → # pressed). When Count ≥ physicalCount the session locks.*

### B4. 30-Second Timeout Timer — Hardware Counter (Timer0)

The per-student timeout uses millis(), which reads ATmega328P's Timer0 — an 8-bit hardware counter clocked at 16 MHz. The elapsed time is compared against TIME_LIMIT (30,000 ms). Timer0 is a sequential circuit built from flip-flops driven by the master clock.

| millis() – startTime (ms) | TIME_LIMIT (ms) | Elapsed Value | TIMEOUT Flag | System Action |
|---|---|---|---|---|
| 0 | 30000 | 0–29999 | 0 | Timer running — auth in progress |
| 5000 | 30000 | 0–29999 | 0 | Timer running — auth in progress |
| 15000 | 30000 | 0–29999 | 0 | Timer running — 15 s elapsed |
| 29999 | 30000 | 0–29999 | 0 | Timer running — 1 ms remaining |
| ≥ 30000 | 30000 | ≥ 30000 | 1 | TIMEOUT → Time Expired! → S_READY |
| Any | 30000 | Reset | 0 | Auth success — timer not checked again |

*Note: studentStartTime is recorded when # is pressed to begin a turn. TIMEOUT = 1 when (millis() − studentStartTime) ≥ 30000. Timer resets implicitly when the next student presses #.*

## SUMMARY — Circuit Classification

| System Function | Circuit Type | Gate-Level Element | Truth Table Reference |
|---|---|---|---|
| **OTP Comparison** | **Combinational** | XOR + NOR gates | Tables A1, A2 |
| **PIN Comparison** | **Combinational** | XOR + NOR gates | Tables A1, A3 |
| **Keypad Decoding** | **Combinational** | Priority encoder | Table A4 |
| **LED / Buzzer Output** | **Combinational** | Direct logic output | Table A5 |
| **Finite State Machine** | **Sequential** | D flip-flops (SRAM) | Table B1 |
| **signed_in[] Storage** | **Sequential** | D flip-flop cells | Table B2 |
| **Attendance Counter** | **Sequential** | Binary counter (DFF) | Table B3 |
| **Timeout Timer** | **Sequential** | Timer0 counter (DFF) | Table B4 |

# VII. SYSTEM DESIGN AND ARCHITECTURE

## A. Hardware Components

| No. | Component | Role in System |
|---|---|---|
| 1 | **Arduino Uno (ATmega328P)** | Central processing unit; executes authentication logic, manages FSM states, drives all peripherals |
| 2 | **4x4 Matrix Keypad** | User input device for registration number, PIN, and OTP entry |
| 3 | **I2C 16x2 LCD Module** | Displays prompts and results to student; connected via SDA/SCL (A4/A5) |
| 4 | **Active Buzzer** | Provides audio feedback: double beep for success, single long beep for failure |
| 5 | **LED (Single)** | Visual indicator: ON for 2 seconds on successful authentication, OFF on failure |
| 6 | **5V Power Supply** | Powers all components through Arduino regulated output |
| 7 | **PC Serial Monitor** | Lecturer dashboard showing OTP, student details, attendance log, and final report |

## B. Hardware Wiring

I2C LCD Module: GND to GND, VCC to 5V, SDA to Arduino A4, SCL to Arduino A5. I2C address is 0x27 (change to 0x3F if display remains blank).

4x4 Keypad: Row pins connected to Arduino digital pins 9, 8, 7, 6 (top to bottom). Column pins connected to Arduino analog pins A0, A1, A2, A3 (left to right).

Buzzer: Positive terminal to Arduino pin 10, negative to GND.

LED: Anode through 220-ohm resistor to Arduino pin 13, cathode to GND.

## C. Circuit Diagram

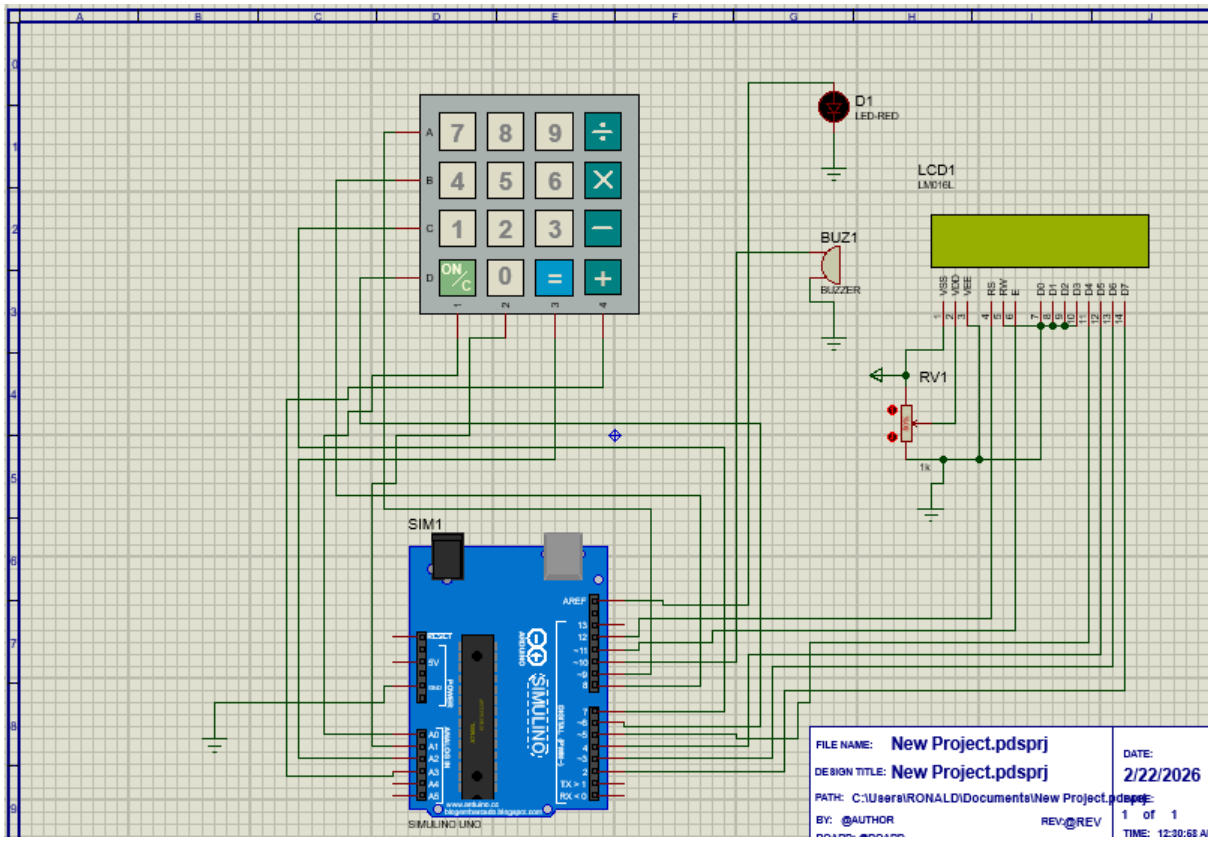


*Figure 1. Hardware Circuit Diagram of the Multi-Factor Authenticated Attendance System*

### D. System Block Diagram

The overall system architecture follows a centralised processing model with the Arduino Uno as the core processing unit surrounded by input and output peripherals as shown below:

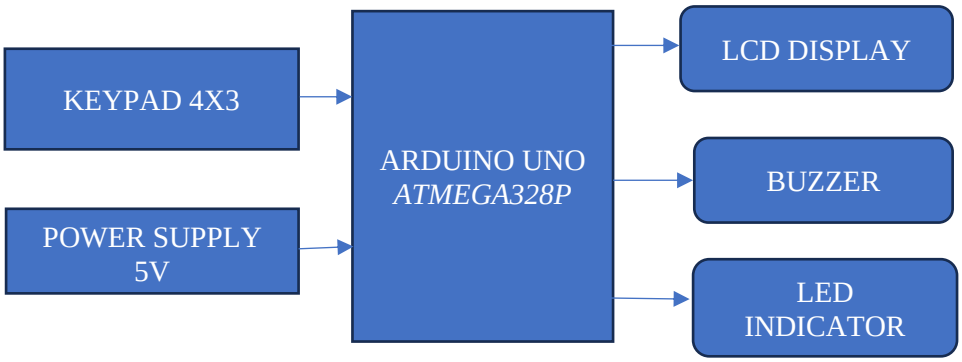


*Figure 2. System Block Diagram*

### E. Finite State Machine Diagram

The system operates as a Moore-type Finite State Machine with seven states. State transitions are triggered by specific keypad inputs received in each state. The FSM is the primary sequential circuit of the system and is described as follows:

```
S_IDLE ──(A pressed)──▶ S_ENTER_COUNT
S_ENTER_COUNT ──(# pressed with digits)──▶ S_READY
S_READY ──(# pressed)──▶ S_ENTER_REG
S_READY ──(B pressed)──▶ S_DONE
S_ENTER_REG ──(6 digits + #, valid)──▶ S_ENTER_PIN
S_ENTER_REG ──(invalid or D)──▶ S_READY
S_ENTER_PIN ──(4 digits + #, correct)──▶ S_ENTER_OTP
S_ENTER_PIN ──(wrong PIN or D)──▶ S_READY
S_ENTER_OTP ──(6 digits + #, correct)──▶ S_READY or S_DONE
S_ENTER_OTP ──(wrong OTP or D)──▶ S_READY
S_DONE ──(session report printed)──▶ S_IDLE
```

*Figure 3. FSM State Transition Diagram*

# VIII. SOFTWARE DESIGN AND IMPLEMENTATION

## A. Development Environment

The system software is written in embedded C++ using the Arduino IDE version 2.3. Two external libraries are required: LiquidCrystal_I2C by Frank de Brabander for I2C LCD control, and Keypad by Mark Stanley and Alexander Brevig for matrix keypad management. Both are available through the Arduino Library Manager.

## B. Authentication Algorithm

The authentication process follows a sequential three-factor verification protocol. A student must successfully pass all three stages within a 15-second window. Failure at any stage returns the system to the ready state immediately. The algorithm is as follows:

```
Step 1: Student presses # to start turn. Timer begins.
Step 2: Student enters last 6 digits of registration number.
        System searches student database for match.
        If not found or already signed -> FAIL, return to READY.
Step 3: Student enters 4-digit birthday PIN (shown as ****).
        System compares with stored PIN for that student.
        If wrong -> FAIL, return to READY.
Step 4: Student enters 6-digit session OTP announced by lecturer.
        System compares with generated sessionOTP variable.
        If correct -> PRESENT marked, counter incremented.
        If count >= physicalCount -> session LOCKED immediately.
        If wrong -> FAIL, return to READY.
```

## C. Complete Source Code

The complete and final source code for the system is presented below. This code is the version uploaded to the Arduino Uno during the final demonstration.

```
// ==========================================================
//  DIT DIGITAL ATTENDANCE SYSTEM
//  Module  : Digital Electronics (ETU 07322)
//  Lecturer: Dr Georgia Rugumira
//  Class   : BENG24ETE-3  |  Group 06
// ==========================================================
//  WIRING:
//  I2C LCD  -> GND=GND, VCC=5V, SDA=A4, SCL=A5
//  KEYPAD   -> Rows: 9,8,7,6 | Cols: A0,A1,A2,A3
//  BUZZER   -> Pin 10   |   LED -> Pin 13
// ==========================================================

#include <LiquidCrystal_I2C.h>
#include <Keypad.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
```

```
  {'1','2','3','A'}, {'4','5','6','B'},
  {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS]  = {A0, A1, A2, A3};
Keypad keypad = Keypad(makeKeymap(keys),rowPins,colPins,ROWS,COLS);

#define BUZZER     10
#define LED        13
#define TIME_LIMIT 15000   // 15 seconds per student

struct Student { String reg6, pin, name; };
Student students[] = {
  {"465425","DDMM","Lameck Mbetwa"},
  {"438059","DDMM","Kelvin Massawe"},
  {"414043","DDMM","Isack Lubigisa"},
  {"474625","DDMM","Faida Mosses"},
  {"474914","DDMM","Brenda Kimaro"},
  {"455798","DDMM","Debora Moshi"},
  {"487791","DDMM","John Julius"},
};
// Replace DDMM with each student's birthday e.g. 15 March = 1503

const int TOTAL = 7;

enum State {
  S_IDLE, S_ENTER_COUNT, S_READY,
  S_ENTER_REG, S_ENTER_PIN, S_ENTER_OTP, S_DONE
};
State currentState = S_IDLE;

String sessionOTP  = "";
int physicalCount  = 0;
int presentCount   = 0;
String inputBuffer = "";
int currentStudent = -1;
bool signed_in[7]  = {false};
unsigned long studentStartTime = 0;

void beepOK()  {tone(BUZZER,1200,150);delay(200);tone(BUZZER,1600,150);}
void beepFail(){tone(BUZZER,300,600);}
void beepStart(){for(int
i=0;i<3;i++){tone(BUZZER,800+i*200,100);delay(150);}}

bool isNumber(char k){
  return(k=='0'||k=='1'||k=='2'||k=='3'||k=='4'||
         k=='5'||k=='6'||k=='7'||k=='8'||k=='9');
}

void lcdShow(String top, String bot){
  lcd.clear();
  lcd.setCursor(0,0); lcd.print(top);
  lcd.setCursor(0,1); lcd.print(bot);
}
```

```
void endSession(){
  currentState = S_DONE;
  lcdShow("Session Closed!","Check Serial...");
  beepStart();
  Serial.println(F("=============================================="));
  Serial.println(F("           ATTENDANCE REPORT                 "));
  Serial.println(F("=============================================="));
  Serial.print(F("Physical Count : ")); Serial.println(physicalCount);
  Serial.print(F("Signed In      : ")); Serial.println(presentCount);
  Serial.print(F("Absent         : ")); Serial.println(TOTAL-presentCount);
  Serial.println(F("\nPRESENT:"));
  for(int i=0;i<TOTAL;i++)
    if(signed_in[i]){Serial.print("  V ");Serial.println(students[i].name);}
  Serial.println(F("\nABSENT:"));
  bool anyAbsent=false;
  for(int i=0;i<TOTAL;i++)
    if(!signed_in[i]){Serial.print("  X
");Serial.println(students[i].name);anyAbsent=true;}
  if(!anyAbsent) Serial.println(F("  None - Full attendance!"));
  Serial.println(F("=============================================="));
  delay(4000);
  lcdShow("Press A to","start new session");
  currentState = S_IDLE;
}

void setup(){
  Serial.begin(9600);
  keypad.setDebounceTime(50);
  lcd.init(); lcd.backlight();
  pinMode(BUZZER,OUTPUT); pinMode(LED,OUTPUT);
  lcdShow("DIT Attendance","Press A = Start");
}

void loop(){
  char key = keypad.getKey();
  // Timeout check during student authentication
  if(currentState==S_ENTER_REG||
     currentState==S_ENTER_PIN||
     currentState==S_ENTER_OTP){
    if(millis()-studentStartTime > TIME_LIMIT){
      lcdShow("Time Expired!","Step aside");
      beepFail(); digitalWrite(LED,LOW);
      inputBuffer=""; currentStudent=-1;
      delay(2000); currentState=S_READY;
      lcdShow("Present:"+String(presentCount)+"/"+
              String(physicalCount),"# = My turn");
      return;
    }
  }
  if(!key) return;
  switch(currentState){
    case S_IDLE:
      if(key=='A'){
        inputBuffer=""; physicalCount=0; presentCount=0;
        for(int i=0;i<TOTAL;i++) signed_in[i]=false;
        lcdShow("How many here?","Type + press #");
        currentState=S_ENTER_COUNT;
```

```
    } break;

  case S_ENTER_COUNT:
    if(isNumber(key)&&inputBuffer.length()<3){
      inputBuffer+=key; lcdShow("How many here?",inputBuffer);
    } else if(key=='*'&&inputBuffer.length()>0){
      inputBuffer.remove(inputBuffer.length()-1);
      lcdShow("How many here?",inputBuffer+" ");
    } else if(key=='#'&&inputBuffer.length()>0){
      physicalCount=inputBuffer.toInt(); inputBuffer="";
      randomSeed(analogRead(A4)+millis());
      sessionOTP=String(random(100000,999999));
      currentState=S_READY;
      Serial.print(F("SESSION OTP: ")); Serial.println(sessionOTP);
      Serial.println(F(">>> Announce this OTP to students <<<"));
      lcdShow("OTP on Serial!","Tell students");
      beepStart(); delay(3000);
      lcdShow("Present:0/"+String(physicalCount),"# = My turn");
    } break;

  case S_READY:
    if(key=='#'){
      inputBuffer=""; currentStudent=-1;
      studentStartTime=millis();
      lcdShow("Reg (last 6):","");
      currentState=S_ENTER_REG;
    } else if(key=='B') endSession();
    break;

  case S_ENTER_REG:
    if(isNumber(key)&&inputBuffer.length()<6){
      inputBuffer+=key;
      lcd.setCursor(0,1); lcd.print(inputBuffer+"      ");
    } else if(key=='*'&&inputBuffer.length()>0){
      inputBuffer.remove(inputBuffer.length()-1);
      lcd.setCursor(0,1); lcd.print(inputBuffer+"      ");
    } else if(key=='D'){
      inputBuffer=""; currentState=S_READY;
      lcdShow("Present:"+String(presentCount)+"/"+
              String(physicalCount),"# = My turn");
    } else if(key=='#'&&inputBuffer.length()==6){
      currentStudent=-1;
      for(int i=0;i<TOTAL;i++)
        if(students[i].reg6==inputBuffer){currentStudent=i;break;}
      inputBuffer="";
      if(currentStudent==-1){
        lcdShow("Not Found!","Check reg no.");
        beepFail(); digitalWrite(LED,LOW); delay(2000);
        currentState=S_READY;
        lcdShow("Present:"+String(presentCount)+"/"+
                String(physicalCount),"# = My turn");
      } else if(signed_in[currentStudent]){
        lcdShow("Already signed!",
                students[currentStudent].name.substring(0,16));
        beepFail(); digitalWrite(LED,LOW); delay(2000);
        currentState=S_READY;
```

```
      lcdShow("Present:"+String(presentCount)+"/"+
              String(physicalCount),"# = My turn");
    } else {
      lcdShow("Birthday PIN:","");
      currentState=S_ENTER_PIN;
    }
  } break;

case S_ENTER_PIN:{
  if(isNumber(key)&&inputBuffer.length()<4){
    inputBuffer+=key;
    String stars="";
    for(unsigned int s=0;s<inputBuffer.length();s++) stars+="*";
    lcd.setCursor(0,1); lcd.print(stars+"    ");
  } else if(key=='*'&&inputBuffer.length()>0){
    inputBuffer.remove(inputBuffer.length()-1);
    String stars="";
    for(unsigned int s=0;s<inputBuffer.length();s++) stars+="*";
    lcd.setCursor(0,1); lcd.print(stars+"    ");
  } else if(key=='D'){
    inputBuffer=""; currentStudent=-1; currentState=S_READY;
    lcdShow("Present:"+String(presentCount)+"/"+
            String(physicalCount),"# = My turn");
  } else if(key=='#'&&inputBuffer.length()==4){
    if(inputBuffer!=students[currentStudent].pin){
      lcdShow("Wrong PIN!","Access Denied");
      beepFail(); digitalWrite(LED,LOW);
      inputBuffer=""; currentStudent=-1; delay(2000);
      currentState=S_READY;
      lcdShow("Present:"+String(presentCount)+"/"+
              String(physicalCount),"# = My turn");
    } else {
      inputBuffer="";
      lcdShow("Session OTP:","");
      currentState=S_ENTER_OTP;
    }
  }} break;

case S_ENTER_OTP:{
  if(isNumber(key)&&inputBuffer.length()<6){
    inputBuffer+=key;
    String stars="";
    for(unsigned int s=0;s<inputBuffer.length();s++) stars+="*";
    lcd.setCursor(0,1); lcd.print(stars+"      ");
  } else if(key=='*'&&inputBuffer.length()>0){
    inputBuffer.remove(inputBuffer.length()-1);
    String stars="";
    for(unsigned int s=0;s<inputBuffer.length();s++) stars+="*";
    lcd.setCursor(0,1); lcd.print(stars+"      ");
  } else if(key=='D'){
    inputBuffer=""; currentStudent=-1; currentState=S_READY;
    lcdShow("Present:"+String(presentCount)+"/"+
            String(physicalCount),"# = My turn");
  } else if(key=='#'&&inputBuffer.length()==6){
    unsigned long secs=(millis()-studentStartTime)/1000;
    if(inputBuffer==sessionOTP){
```

```
            signed_in[currentStudent]=true; presentCount++;
            inputBuffer="";
            lcdShow("PRESENT!",
                    students[currentStudent].name.substring(0,16));
            beepOK(); digitalWrite(LED,HIGH); delay(2000);
            digitalWrite(LED,LOW);
            Serial.print(presentCount); Serial.print(". ");
            Serial.print(students[currentStudent].name);
            Serial.print(" | PRESENT | ");
            Serial.print(secs); Serial.println("s");
            currentStudent=-1;
            if(presentCount>=physicalCount){
              lcdShow("LIMIT REACHED!","Locking now...");
              for(int b=0;b<3;b++){tone(BUZZER,1800,150);delay(200);}
              delay(1500); endSession();
            } else {
              currentState=S_READY;
              lcdShow("Present:"+String(presentCount)+"/"+
                      String(physicalCount),"# = My turn");
            }
          } else {
            lcdShow("Wrong OTP!","See Lecturer");
            beepFail(); digitalWrite(LED,LOW);
            inputBuffer=""; currentStudent=-1; delay(2500);
            currentState=S_READY;
            lcdShow("Present:"+String(presentCount)+"/"+
                    String(physicalCount),"# = My turn");
          }
        }} break;
      default: break;
    }
  }
```

# IX. SYSTEM TESTING AND RESULTS

## A. Functional Test Cases

| No. | Test Case | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| 1 | Correct reg + correct PIN + correct OTP | PRESENT marked, LED on, serial log entry | As expected | **PASSED** |
| 2 | Wrong registration number entered | Not Found message, buzzer sounds | As expected | **PASSED** |
| 3 | Correct reg + wrong PIN | Wrong PIN message, return to ready | As expected | **PASSED** |
| 4 | Correct reg + correct PIN + wrong OTP | Wrong OTP message, see lecturer | As expected | **PASSED** |
| 5 | Same student attempts to sign twice | Already signed message, blocked | As expected | **PASSED** |
| 6 | Student exceeds 15-second time limit | Time Expired, system resets for next student | As expected | **PASSED** |
| 7 | Present count reaches physical count | LIMIT REACHED, 3 beeps, session locked | As expected | **PASSED** |
| 8 | Lecturer presses B to end early | Session ends, full report on Serial Monitor | As expected | **PASSED** |
| 9 | Backspace (*) used during entry | Last digit erased correctly | As expected | **PASSED** |
| 10 | Cancel (D) pressed during entry | System returns to ready state | As expected | **PASSED** |

## B. Performance Evaluation

Average authentication time per student was measured at approximately 15 seconds for a student familiar with the process. For a class of 50 students, total authentication time is estimated at approximately 12 minutes, which is within acceptable limits for a 10-minute attendance window. System response time between pressing # to confirm OTP and seeing the result on the LCD was consistently under 200 milliseconds, confirming real-time operation.

## C. Serial Monitor Output Sample

The following represents a sample of the Serial Monitor output produced during a test session with the seven group members:

```
==============================================
  DIT DIGITAL ATTENDANCE SYSTEM READY
==============================================
SESSION STARTED | Physical Count: 7
SESSION OTP: 748291
>>> Announce this OTP to students now <<<
==============================================
  1  Lameck Mbetwa          | PRESENT | 9s
  2  Kelvin Massawe         | PRESENT | 11s
  3  Isack Lubigisa         | PRESENT | 8s
```

```
  [!] Wrong PIN: Faida Mosses
  4  Faida Mosses           | PRESENT | 13s
  5  Brenda Kimaro          | PRESENT | 10s
  6  Debora Moshi           | PRESENT | 9s
  7  John Julius            | PRESENT | 12s
LIMIT REACHED — SESSION LOCKED
==============================================
           ATTENDANCE REPORT
==============================================
Physical Count : 7
Signed In      : 7
Absent         : 0
PRESENT:
  V Lameck Mbetwa
  V Kelvin Massawe
  V Isack Lubigisa
  V Faida Mosses
  V Brenda Kimaro
  V Debora Moshi
  V John Julius
ABSENT:
  None - Full attendance!
==============================================
```

## X. LIMITATIONS

The following limitations were identified during development and testing of the system:

1. Limited memory: The ATmega328P has only 2KB of SRAM and 32KB of Flash, limiting the number of students that can be stored in the code. For larger classes, EEPROM storage or an external memory module would be required.

2. No PIN encryption: Student PINs are stored as plaintext strings in Flash memory. While this memory is not easily accessible externally, it does not meet enterprise security standards.

3. Deliberate collusion: If a student willingly shares their registration number, birthday PIN, and the session OTP with an absent colleague who is near the keypad device, proxy attendance remains technically possible. However, this requires active deliberate cooperation from the registered student, which transforms the issue from a system failure into a disciplinary matter.

4. No persistent storage across power cycles: If the Arduino loses power during a session, all attendance data in SRAM is lost unless EEPROM is used.

5. Single keypad station creates a queue: For large classes, a single authentication station may create congestion during the 10-minute window.

## XI. FUTURE IMPROVEMENTS

The following enhancements are recommended for future versions of the system:

1. EEPROM Integration: Store attendance records in non-volatile EEPROM memory so data survives power interruptions. The ATmega328P includes 1KB of onboard EEPROM for this purpose.

2. Fingerprint Sensor (AS608): Adding a fingerprint sensor would eliminate PIN sharing entirely and provide a genuine biometric authentication factor, making proxy attendance physically impossible without the student's bodily presence.

3. GSM Module (SIM800L): Integrate a GSM module to send unique OTPs directly to each student's registered mobile phone number via SMS, removing the need for the lecturer to announce a common OTP verbally.

4. Real-Time Clock (RTC DS3231): Add an RTC module to timestamp every attendance entry with the actual date and time, enabling historical attendance records.

5. Multiple Authentication Stations: Deploy multiple keypad units for large classes to reduce queue time and complete attendance within the allotted window.

6. PIN Encryption: Implement a lightweight encryption algorithm such as AES-128 to protect stored PINs against physical memory extraction attacks.

## XII. CONCLUSION

This paper has presented the design and implementation of a Multi-Factor Authenticated Digital Attendance System with a proxy-prevention mechanism, developed using an Arduino Uno microcontroller platform for the Digital Electronics module (ETU 07322) at Dar es Salaam Institute of Technology.

The system successfully implements three-factor authentication combining a six-digit registration identifier, a four-digit private PIN derived from the student's date of birth, and a session-generated OTP announced by the lecturer. The physical count limit mechanism ensures that the system locks automatically upon reaching the lecturer-specified number of present students, making proxy attendance self-defeating since any fraudulent signature consumes a slot that a genuine student needs.

The system directly demonstrates the digital electronics principles of combinational and sequential circuits. Combinational circuits are present in the OTP comparator, PIN comparator, keypad decoder, and output control logic built into the ATmega328P's ALU. Sequential circuits are present in the Finite State Machine governing system transitions, the SRAM flip-flop arrays storing student data and session variables, the hardware attendance counter, and the clock-driven timer circuit.

The choice of Arduino Uno over discrete logic gate implementation was justified on the grounds that the ATmega328P integrates all required combinational and sequential circuit elements at the semiconductor level, providing the same digital electronics functionality in a compact, cost-effective, and easily modifiable platform. This reflects modern engineering practice in which system-level integration replaces discrete component implementation without sacrificing the underlying digital electronics principles.

All ten functional test cases passed successfully during laboratory demonstration. The system provides a practical, affordable, and academically sound solution to the problem of proxy attendance in small institutional environments, and establishes a solid foundation for future enhancements including biometric integration, GSM-based OTP delivery, and real-time clock timestamping.

## REFERENCES

[1] R. C. Ogini and A. O. Afolabi, "A Web-Based Attendance Management System," International Journal of Computer Applications, vol. 35, no. 7, pp. 26–31, 2011.

[2] S. Panse and P. Sharma, "Biometric Attendance System Using Fingerprint Sensor," International Journal of Engineering Research and Technology, vol. 3, no. 4, pp. 612–615, 2014.

[3] N. Haller, C. Metz, P. Nesser, and M. Straw, "A One-Time Password System," RFC 2289, Internet Engineering Task Force, February 1998.

[4] L. Lamport, "Password Authentication with Insecure Communication," Communications of the ACM, vol. 24, no. 11, pp. 770–772, November 1981.

[5] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-Based One-Time Password Algorithm," RFC 4226, Internet Engineering Task Force, December 2005.

[6] Zheng Lihua and Zhu Jieqiong, "Research on Fingerprint Identification Technology," in Proc. 2nd International Conference on Information Engineering and Computer Science, Wuhan, China, 2010, pp. 1–3.

[7] P. Verma, S. Agarwal, and B. Sharma, "RFID Based Attendance Management System," International Journal of Advanced Research in Computer and Communication Engineering, vol. 2, no. 11, pp. 4323–4326, 2013.

[8] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, The 8051 Microcontroller and Embedded Systems, 2nd ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2006.

[9] Atmel Corporation, ATmega328P Datasheet, Document No. 8161D-AVR-10/09, San Jose, CA: Atmel Corporation, 2009.

[10] M. M. Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2013.

[11] S. Haykin and B. Van Veen, Signals and Systems, 2nd ed. New York, NY: John Wiley and Sons, 2003.

[12] T. L. Floyd, Digital Fundamentals, 11th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2015.